

PANDUAN IMPLEMENTASI & FLOW KERJA TIM

Urutan Pengerjaan yang Disarankan

Ikuti urutan ini untuk menghindari blocker antar anggota:

Minggu	Focus Area	Kegiatan Utama	PIC
1-2	Fondasi	Repo setup, finalisasi API contract, Figma wireframe, setup Ollama + PostgreSQL, buat KB sintetik awal (10 entri)	Adrian, Ilham, Ahmadhani, Kevin, Rasky
3-4	Core Development	Backend API (auth + chat endpoint), RAG pipeline dasar, Chat UI frontend, knowledge base 35 entri	Ahmadhani, Rasky, Ilham, Kevin
5-6	Integrasi	Connect frontend-backend, integrasi AI engine, eskalasi tiket, dashboard admin v1	Semua + Andri mulai test plan
7	Testing & Fix	Black-box 30 skenario, API testing Postman, bug fixing berdasarkan hasil test	Andri, semua fix bug
8	UAT & Polish	UAT dengan responden, SUS score, UI polish, performance test, finalisasi dokumentasi	Andri, Ilham, Adrian
9	Final	Demo preparation, laporan akhir, presentasi PoC	Adrian koordinasi semua

Contoh Implementasi: Endpoint Chat (Referensi Rasky & Ahmadhani)

```

// backend/src/services/chatService.js
const { OllamaClient } = require('./ollamaClient');
const { vectorSearch } = require('./vectorStore');
const { db } = require('../database');

async function processChat(userId, sessionId, message, imageBase64 = null) {
  // 1. Retrieve relevant KB documents
  const embedding = await OllamaClient.embed(message);
  const topDocs = await vectorSearch(embedding, topK=3);

  // 2. Build context string
  const context = topDocs.map(doc =>
    `[Source: ${doc.title}]\n${doc.content}`
  ).join('\n\n---\n\n');

  const confidence = topDocs.length > 0 ? topDocs[0].score : 0;
  const escalated = confidence < 0.60;

  // 3. Build prompt with RAG context
  const systemPrompt = `Kamu adalah asisten helpdesk teknis PT. Indonesia Epson Industry.
  Jawab pertanyaan karyawan HANYA berdasarkan context di bawah ini.
  Jika tidak ada informasi relevan, katakan tidak tahu dan sarankan eskalasi ke IT Support.
  Gunakan bahasa Indonesia yang jelas dan berikan langkah-langkah yang spesifik.

  CONTEXT:
  ${context}`;

  // 4. Call Ollama LLM
  const reply = escalated
    ? 'Maaf, saya tidak menemukan solusi untuk masalah ini. Tim IT Support akan segera dihubungi.'
    : await OllamaClient.generate(systemPrompt, message);

  // 5. Save to database
  const messageId = `msg_${Date.now()}`;
  await db.chat_logs.create({
    data: { id: messageId, session_id: sessionId, user_id: userId,
      role: 'assistant', content: reply, confidence, escalated,
      kb_sources: topDocs.map(d => d.id) }
  });

  // 6. Create ticket if escalated
  let ticketId = null;
  if (escalated) {
    const ticket = await db.tickets.create({
      data: { id: `tkt_${Date.now()}`, user_id: userId,
        session_id: sessionId, question: message, status: 'open' }
    });
    ticketId = ticket.id;
  }

  return { reply, confidence, sources: topDocs, escalated, ticketId, messageId };
}

```

Implementasi Hybrid AI Router (Rasky - Referensi Kode)

Berikut implementasi lengkap AI Router yang memungkinkan sistem menggunakan Ollama lokal dan API eksternal secara bersamaan:

```

// ai-engine/src/aiRouter.js
// Menentukan engine berdasarkan kompleksitas query

const COMPLEXITY_KEYWORDS = ['kenapa', 'analisis', 'bandingkan', 'rekomendasi',
'jelaskan secara detail', 'apa penyebab'];

function calculateComplexity(message) {
  const lower = message.toLowerCase();
  const keywordHits = COMPLEXITY_KEYWORDS.filter(k => lower.includes(k)).length;
  const lengthScore = Math.min(message.length / 300, 1); // normalized 0-1
  return Math.min((keywordHits * 0.25) + (lengthScore * 0.5), 1.0);
}

async function selectAndGenerate(systemPrompt, userMessage, imageBase64 = null) {
  const complexity = calculateComplexity(userMessage);
  const hasImage = !!imageBase64;
  let engineUsed = '';

  try {
    if (hasImage) {
      // Gambar → wajib API eksternal multimodal
      engineUsed = 'openai-gpt4o';
      return { reply: await generateOpenAI(systemPrompt, userMessage, imageBase64,
'gpt-4o'), engineUsed };
    } else if (complexity >= 0.5) {
      // Kompleks → API eksternal (lebih pintar)
      engineUsed = 'openai-gpt4o-mini';
      return { reply: await generateOpenAI(systemPrompt, userMessage, null,
'gpt-4o-mini'), engineUsed };
    } else {
      // Sederhana → Ollama lokal (gratis & privat)
      engineUsed = 'ollama-local';
      return { reply: await generateOllama(systemPrompt, userMessage), engineUsed };
    }
  } catch (err) {
    // Fallback ke lokal jika API eksternal error/timeout
    console.warn(`Engine ${engineUsed} failed, falling back to Ollama:`,
err.message);
    engineUsed = 'ollama-local-fallback';
    return { reply: await generateOllama(systemPrompt, userMessage), engineUsed };
  }
}

// --- Ollama (Lokal) ---
async function generateOllama(systemPrompt, userMessage) {
  const res = await fetch('http://localhost:11434/api/chat', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      model: 'llama3.2', // atau model lain yang di-pull
      messages: [
        { role: 'system', content: systemPrompt },
        { role: 'user', content: userMessage }
      ],
      stream: false
    })
  });
  const data = await res.json();
  return data.message.content;
}

// --- OpenAI API (Eksternal) ---
async function generateOpenAI(systemPrompt, userMessage, imageBase64 = null, model =
'gpt-4o-mini') {
  const userContent = imageBase64

```

```

    ? [
      { type: 'text', text: userMessage },
      { type: 'image_url', image_url: { url: `data:image/jpeg;base64,${imageBase64}` } }
    ]

    : userMessage;

const res = await fetch('https://api.openai.com/v1/chat/completions', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${process.env.OPENAI_API_KEY}`
  },
  body: JSON.stringify({
    model,
    messages: [
      { role: 'system', content: systemPrompt },
      { role: 'user', content: userContent }
    ],
    max_tokens: 1000
  })
});
const data = await res.json();
return data.choices[0].message.content;
}

module.exports = { selectAndGenerate };

```

Di chatService.js, ganti panggilan LLM langsung dengan AI Router:

```

// backend/src/services/chatService.js (potongan)
const { selectAndGenerate } = require('../../ai-engine/src/aiRouter');

// ...setelah retrieval RAG...
const { reply, engineUsed } = escalated
  ? { reply: 'Maaf, saya tidak menemukan solusi...', engineUsed: 'none' }
  : await selectAndGenerate(systemPrompt, message, imageBase64);

// Simpan engineUsed ke chat_logs untuk monitoring & analisis biaya
await db.chat_logs.create({ data: { ..., engine_used: engineUsed } });

```

CHECKLIST MILESTONE & KRITERIA KEBERHASILAN

Gunakan checklist ini untuk memastikan semua kriteria keberhasilan dari LK01 terpenuhi:

Kriteria	Target LK01	PIC
Chatbot merespons pertanyaan teknis dalam bahasa alami	Fungsional	Rasky + Ahmadhani
Jawaban sesuai knowledge base (tidak halusinasi)	Akurasi >= 80%	Rasky + Kevin
Black-box testing 30 skenario	Pass rate >= 80%	Andri + Rasky

Eskalasi tiket otomatis jika chatbot tidak bisa jawab	Berfungsi	Ahmadhani + Rasky
Riwayat percakapan tersimpan 100%	Berfungsi	Ahmadhani
UAT dengan kuesioner SUS	Skor >= 70	Andri + Ilham
Dashboard monitoring statistik masalah	Berfungsi	Ilham + Kevin
Data & KB tersimpan di server internal (on-premise)	Verified	Adrian + Ahmadhani
AI engine berjalan (lokal, API, atau hybrid)	Minimal 1 mode aktif	Rasky
Response time chatbot	< 3 detik (atau < 5s untuk LLM)	Andri (performance test)
PoC live demo tanpa crash	Stable demo	Semua

Catatan Penting

Data Produksi

Seluruh data knowledge base menggunakan data SINTETIK. Tidak ada data asli PT. Epson yang digunakan. Fokus proyek adalah membuktikan ARSITEKTUR sistem bekerja dengan benar — chatbot merespons sesuai knowledge base yang ada.

Arsitektur AI Hybrid

Sistem TIDAK wajib full on-premise untuk AI engine. Pilihan tersedia: (1) Ollama Lokal saja — gratis, privat, cocok untuk MVP dan pertanyaan standar. (2) API Eksternal saja — kualitas lebih tinggi, mudah setup, ada biaya per-token. (3) Hybrid (direkomendasikan) — Ollama untuk pertanyaan sederhana/murah, API eksternal untuk pertanyaan kompleks/gambar. Data sensitif (KB, chat logs, users) TETAP di server internal.

Koordinasi

Setiap perubahan pada API contract (Bab 3) WAJIB dikomunikasikan ke seluruh tim via GitHub Issues atau group chat sebelum diimplementasikan. Perubahan mendadak pada interface akan memblokir pekerjaan anggota lain.